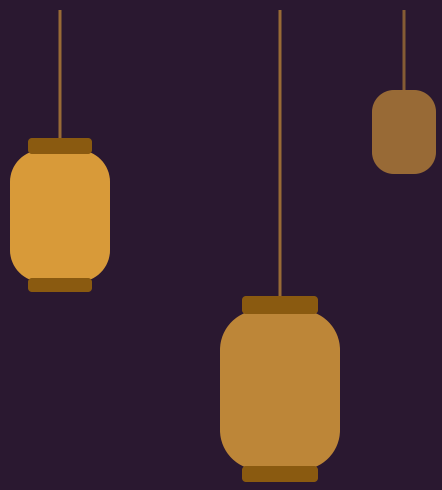
VN-ENGINE · A BEGINNER'S GUIDE



# Your first game

## Make a visual novel in Studio, step by step

No game-making or programming needed. In about thirty minutes you make *The Night Bakery*: a street at midnight, a baker named Mika, a choice and two endings. Then you dress it up, give it choices that matter, try the tools for bigger games, and build it for other people to play.

November 2026 · Every step was done in Studio while this guide was written.

INSIDE

# What you will make

Each step says what to do, then what you should see. New words are explained when they first appear, and they are all in the glossary at the end.


**1** Your first game

**2** Make it yours

**3** Choices that matter

**4** Bigger games

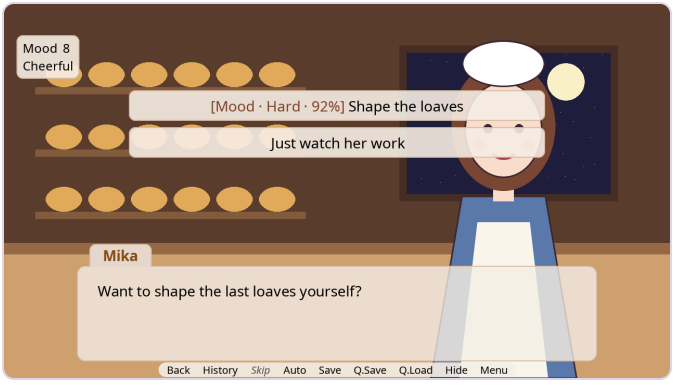
**5** Test and share

**?** Troubleshooting and glossary

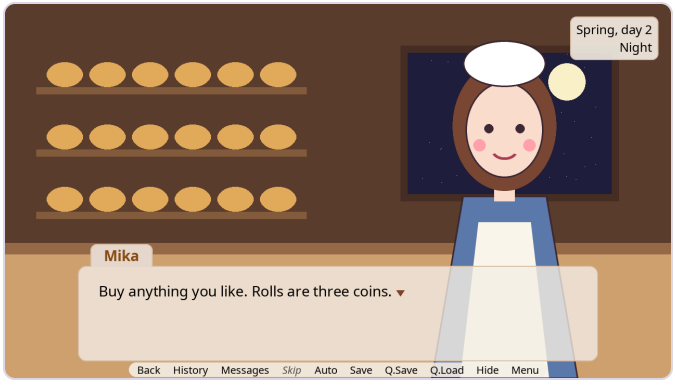




The Night Bakery: Mika's first line.




A choice with a dice roll (Part 3).




A shop on the second night (Part 4).

This guide is for you if you have never made a game. You will make a short visual novel called *The Night Bakery*: a street at midnight, a baker named Mika, a choice, and a few endings. Every step was done in Studio, the editor that comes with vn-engine, while this guide was written.

Each step says what to do, then what you should see. If what you see is different, look in Troubleshooting at the end. New words are explained the first time they appear, and all of them are in the Glossary.

- **Part 1, Your first game** (about 30 minutes): a game you can play from start to end.
- **Part 2, Make it yours**: colors, the title screen, music, effects.
- **Part 3, Choices that matter**: choices the game remembers, and more endings.
- **Part 4, Bigger games**: story time, money and a shop, letters, the codex, minigames, chapters.
- **Part 5, Test and share**: finding mistakes, and making a game other people can play.

BEFORE YOU START

# Before you start

You need:

- A Windows computer.
- **Python** 3.12 or newer, from python.org. Python is the program that runs vn-engine. When the installer asks, tick **Add python.exe to PATH**.
- The **vn-engine folder**, unzipped somewhere you can find it, such as your Desktop.
- **Pictures** for your game: a few pictures of one character, and two backgrounds.
  - A **background** is a picture of a place, 1280 × 720 pixels, as a PNG or JPG file.
  - A **character picture** is a PNG of the character with a see-through background, about 400 × 680 pixels. Make one picture for each face you want: happy, sad, surprised.
  - Name each file in lower case, with no spaces: `happy.png`, `sad.png`, `surprised.png`, `bakery.png`, `street_night.png`. The file name becomes the name you use in your story.

Put your pictures in a folder of their own, such as `My Pictures/night-bakery`. Studio copies them into your game when you add them.

Then install the one extra piece vn-engine needs. Open the Start menu, type `cmd` and press Enter. In the black window that opens (a **terminal**), type:

```
pip install "pygame-ce>=2.5.2,<2.6"
```

You should see "Successfully installed pygame-ce" at the end.

PART 1

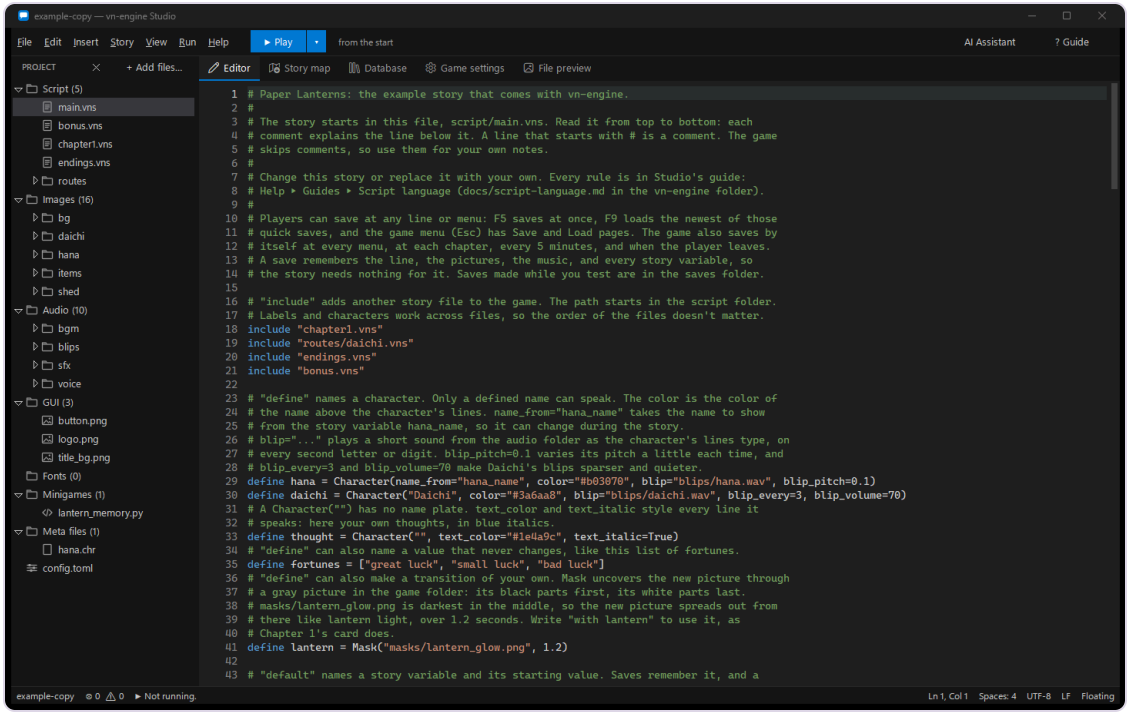
# Your first game

## Step 1: Open Studio

1. Open the vn-engine folder in File Explorer.
2. Click the address bar at the top, type `cmd` and press Enter. A terminal opens in that folder.
3. Type this and press Enter:

```
python -m engine edit
```

You should see Studio open on *Paper Lanterns*, the example game that comes with vn-engine. You can look around it any time. Next you make a game of your own.



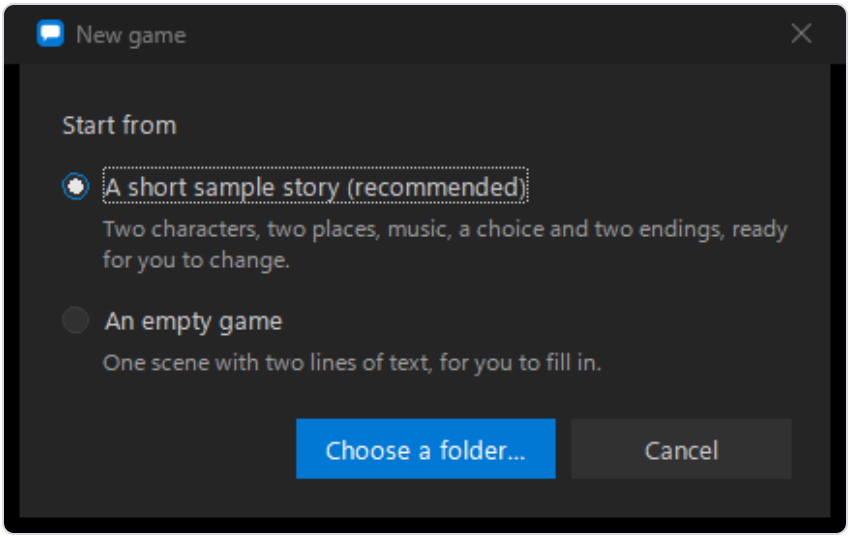

Studio open on Paper Lanterns, the example game.

## Step 2: Make a new game

1. In Studio, choose **File ▸ New game...**.
2. The **New game** window asks where to **Start from**. Keep **A short sample story (recommended)** picked. It gives you two characters, two places, music, a choice and two endings to learn from.
3. Click **Choose a folder...**. Pick an empty folder, or type a new folder name such as `night-bakery`, and confirm.

You should see a second Studio window open on your new game. The first window stays open; you can close it. On the right is the **Guide** pane, "Your first game", with seven steps and "2 of 7 done". The steps tick themselves as you work. This guide follows the same steps and goes further.

A **genre kit** (a ready-made look for a romance, horror story, mystery and so on) is not part of New game. You pick one in Part 2, and you can change it any time.




File ▸ New game... asks where to start from.

## Step 3: Play the sample

1. Click the arrow ▾ next to ▶ **Play** at the top, and choose **Play from the start** (or press Shift+F5).

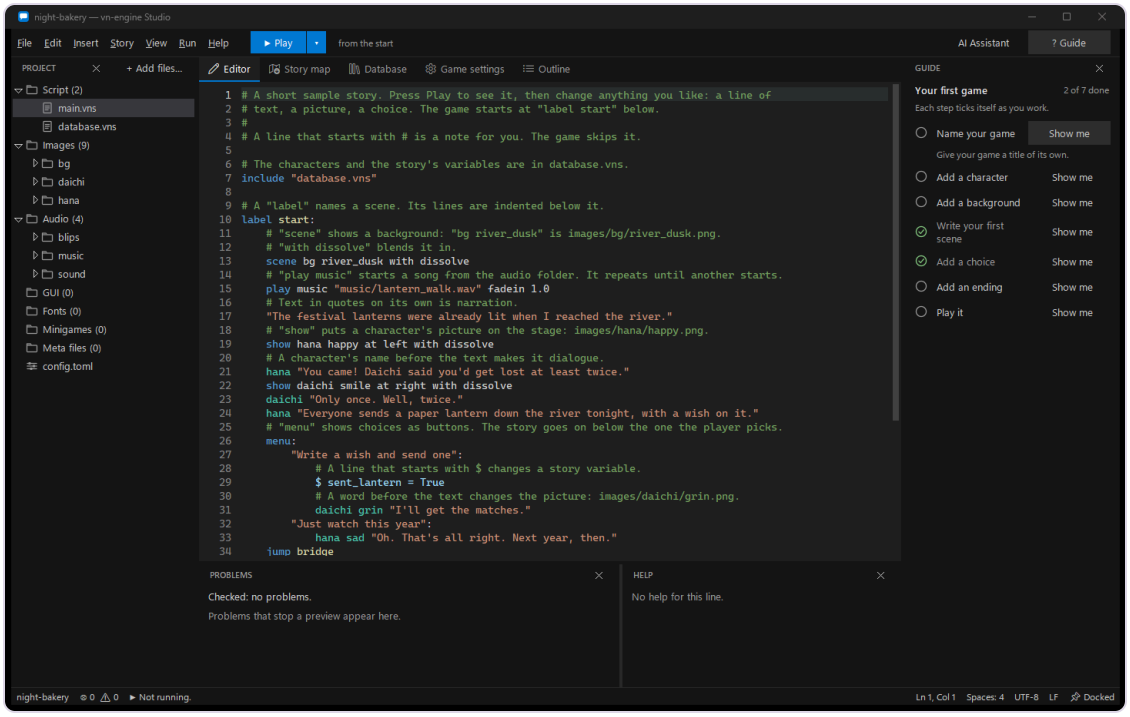
You should see the game open next to Studio, on a title screen with **New Game**, **Load**, **Settings** and **Quit**. Click **New Game**. Click, or press Space or Enter, to go to the next line. When the choice appears, click one.

1. Close the game with ■ **Stop** (where Play was), or close its window.




The sample story playing. Click or press Space for the next line.

## Step 4: A tour of Studio



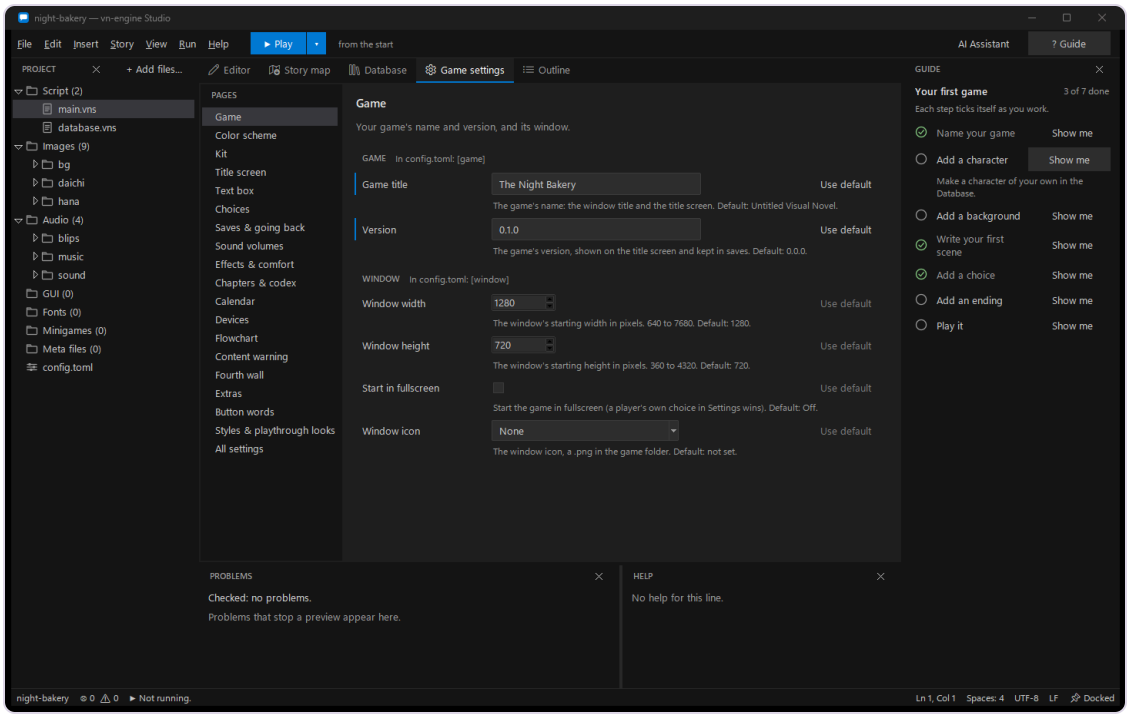

A new game in Studio: Project on the left, the Editor in the middle, Problems and Help below, the Guide on the right.

- **Project** (left): your game's files. **Script** holds the story files. A story file ends in `.vns` and is plain text. **Images** holds pictures, **Audio** holds music and sounds. `config.toml` holds the game's settings. **+ Add files...** copies files into the folder you selected.
- **Editor** tab (Ctrl+1): the text of the story file you clicked. This text is the **script**: the story written as lines Studio and the game can read.
- **Story map** tab (Ctrl+2): each scene drawn as a box, with arrows to the scenes it leads to. Click a box to open that scene beside the map, with two tabs: **Events**, a list of what happens in the scene, one row per line, and **Script**, the same lines as text.
- **Database** tab (Ctrl+3): forms for your characters, pictures, sounds, variables and more. Studio writes the script lines for you.
- **Game settings** tab (Ctrl+4): forms for the game's title, colors, title screen, text box and more.
- **Outline** tab: a list of chapters, endings and other parts of your story.
- **Problems** (bottom left): mistakes Studio finds as you type, each with the line it is on.
- **Help** (bottom right): what the line under the cursor does, with an example.
- **▶ Play** (top): plays the game. The words beside it say where it starts, such as "from the start".
- **? Guide** (top right): shows or hides the Guide pane. **Help ▸ Guides** opens this guide and the others.

## Step 5: Name your game

1. Click the **Game settings** tab. The **Game** page is open.
2. Click in **Game title**, select the folder name that is there, type `The Night Bakery` and press Enter.
3. Choose **File ▸ Save all** (Ctrl+Shift+S).

You should see "Name your game" ticked in the Guide. The title shows on the game's window and title screen.

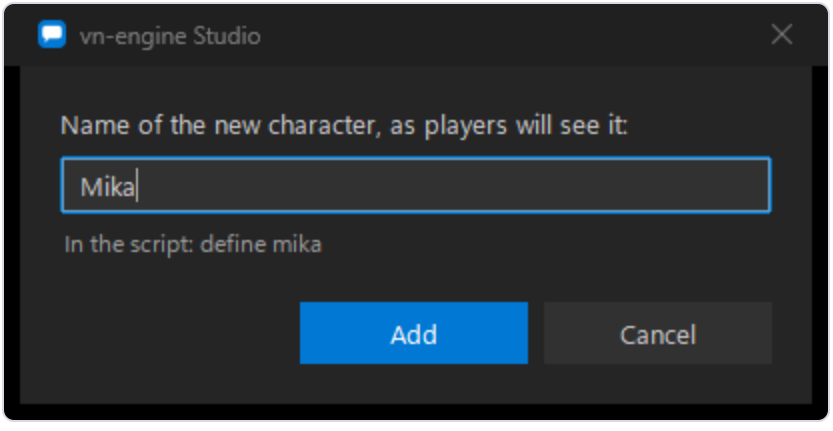



Game settings ▸ Game with the new title. The Guide ticks the step after the next check.

## Step 6: Add a character with pictures

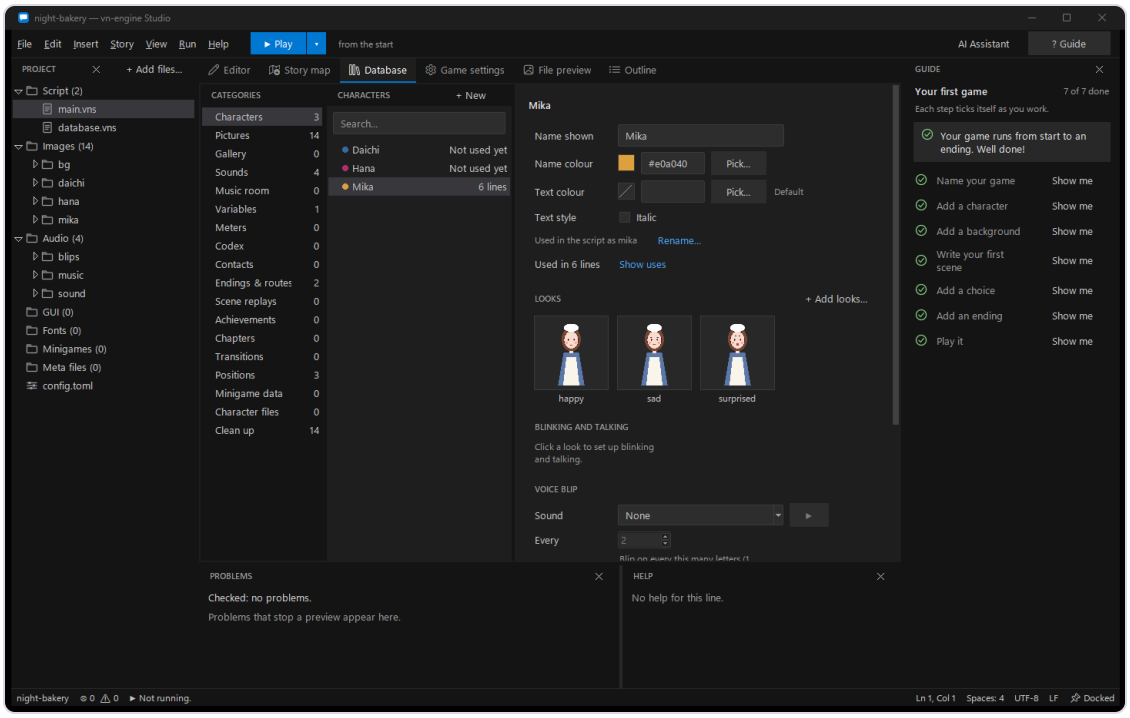
A **character** is someone who speaks in your story.

1. Click the **Database** tab, then **Characters** on the left.
2. Click **+ New**. Studio asks for the "Name of the new character, as players will see it". Type `Mika`. Under the box it says "In the script: define mika": `mika` is the name you type in the story. Click **Add**.
3. Mika's form opens. Click **Pick...** beside **Name colour** and choose a dark color, such as a dark brown. A light color is hard to read on a light name tab.
4. Click **+ Add looks...**, pick `happy.png`, `sad.png` and `surprised.png` from your pictures folder, and click **Open**.
5. Choose **File ▸ Save all**.




+ New asks for the character's name. The grey line shows the name to use in the script.

You should see "Added 3 files.", and Studio shows the last picture on the **File preview** tab with its name, such as "Image name: mika surprised". Each picture is a **look** of Mika: `mika happy`, `mika sad`, `mika surprised`. Back in **Database** ▸ **Characters** ▸ **Mika**, the three looks are under **Looks**. The Guide ticks "Add a character".



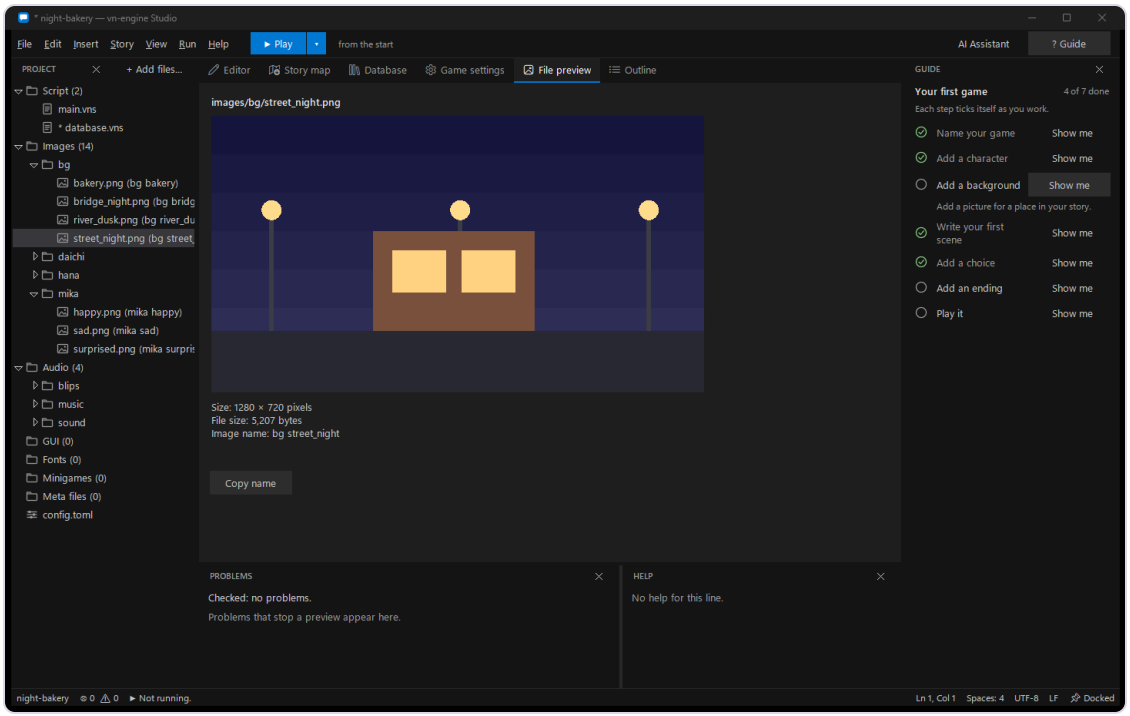

Database ▸ Characters ▸ Mika, with her three looks.

If a file name has spaces or capitals, Studio offers a better one, for example "`Mika Happy.png` can't be an image name (spaces). Use `mika_happy.png`?". Type the look's name (`happy.png`) and click **Add**.

## Step 7: Add backgrounds

1. In the **Database**, click **Pictures**, then **+ New** ▸ **Add backgrounds...**.
2. Pick `bakery.png` and `street_night.png` and click **Open**.

You should see "Added 2 files.". The pictures are now `bg bakery` and `bg street_night`. The first word, `bg`, is the picture's **tag**: every background has the tag `bg`, and each of Mika's looks has the tag `mika`. The Guide ticks "Add a background".




After Add backgrounds..., File preview shows the last picture and its image name, bg street_night.

## Step 8: Write your first scene

A **scene** is one part of the story, such as "at the bakery". In the script, a scene starts with a **label** line: `label` and the scene's name, ending in a colon. The lines of the scene go under it, moved four spaces to the right (**indented**). The game always begins at the scene called `start`.

1. Click `main.vns` under **Script** in the Project panel. It opens in the Editor.
2. Lines that start with `#` are notes for you; the game skips them. Find the line `label start:`. Select everything from that line to the end of the file and delete it. Keep `include "database.vns"` above it: that line adds the file where Mika lives.
3. Type this. After a line that ends in a colon, Enter indents the next line for you; Shift+Tab moves a line back to the left.

```
label start:
    scene bg street_night with fade
    "It is almost midnight. Every shop on the street is dark except one."
    "A small sign in the window says: OPEN UNTIL THE BREAD IS GONE."
    scene bg bakery with dissolve
    show mika happy with dissolve
    mika "Oh! A customer! Come in, come in."
    mika "I'm Mika. I bake at night, when the oven is warm and the town is quiet."
    mika surprised "Wait, you're shivering. Did you walk all the way here?"
```

1. Press Ctrl+S to save.

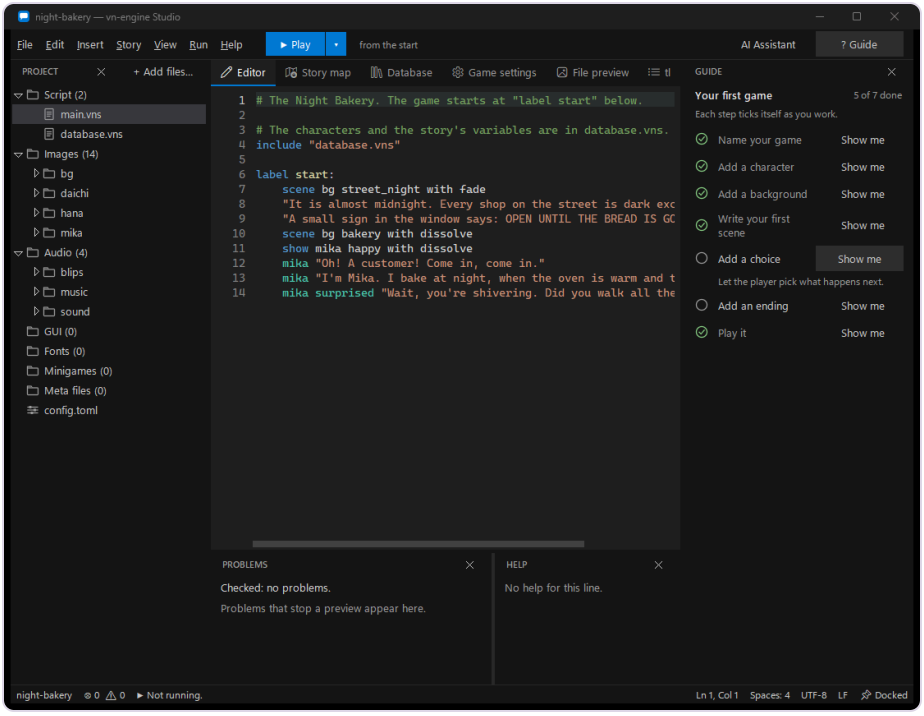
What each line does:

- `scene bg street_night` shows a background and clears everything else off the stage. `with fade` goes through black; `with dissolve` blends one picture into the next. These are **transitions**.

- Text in quotes on its own is **narration**: the story telling the player what happens.
- `show mika happy` puts Mika on the stage, in the middle. `hide mika` takes her away.
- `mika "…"` is **dialogue**: Mika's name shows above the text.
- `mika surprised "…"` changes Mika to her surprised look as she says the line. This is how you change a character's **expression**.

You should see "Checked: no problems." in Problems. The Guide ticks "Write your first scene" (the sample already had one, so it may be ticked already).

While you type, **Ctrl+Space** lists the names that fit, such as your characters and pictures, and the **Help** pane explains the line under the cursor.

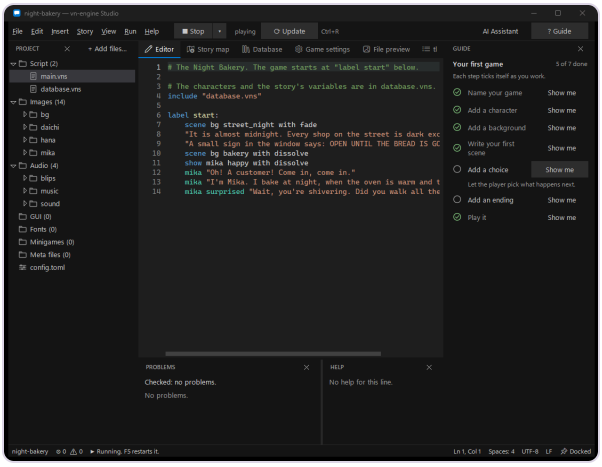



The first scene in the Editor.

## Step 9: Play it

1. Put the cursor on any line of your scene and press ▶ **Play** (F5). The words beside it say "from start".

You should see the street at night, your two lines of narration, then the bakery with Mika and her name above her lines. Her face changes on the last line.



While the game runs, Play becomes Stop and Update, and the status bar says Running.




Mika's first line in the game.

1. Press ■ **Stop** (Ctrl+F2).

After her last line, the game goes back to the title screen. The story has nowhere else to go yet.

## Step 10: Add a choice with two branches

A **choice** (called a **menu** in the script) shows buttons and lets the player decide what happens next. Each button can go to another scene with `jump`.

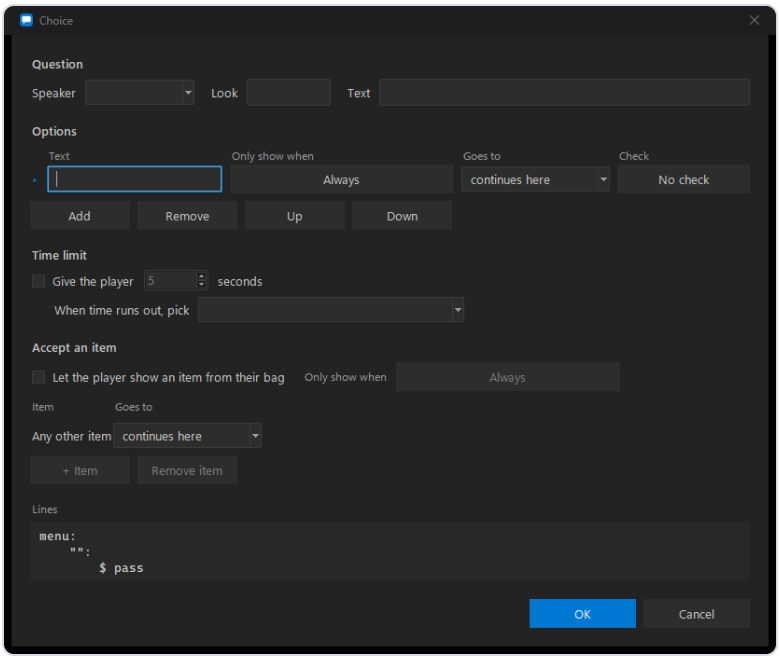
1. Add these lines at the end of the `start` scene, indented like the lines above them:

```
    menu:
        mika "Can I get you something?"
        "Ask for the warmest bread she has":
            jump warm_bread
        "Say you were just looking":
            jump just_looking
```

The first line under `menu:` is the question, shown while the player chooses. Each quoted line ending in a colon is a button, and the lines indented under it run when the player picks it.

You should see two errors in Problems, such as "there is no label named "warm_bread"". That is expected: those scenes don't exist yet. You make them in the next step.

You can also make a choice without typing: on the **Story map**, click the `start` box, then in **Events** click **Insert...**, open **Choices** and pick **Menu**. The **Choice** form has the **Question**, and for each option its **Text**, **Only show when** and **Goes to**.




The Choice form: the question, each option and where it goes.

## Step 11: Add endings

An **ending** finishes the story: the game shows a card with the ending's name, remembers that the player reached it, and goes back to the title screen.

1. Add these two scenes at the very end of `main.vns`, with `label` at the left edge:

```
label warm_bread:
    mika happy "The warmest bread? I have just the thing."
    "She sets a round loaf on the counter. Steam curls up from the crust."
    ending warm "Good End" "A Warm Loaf"

label just_looking:
    mika sad "Oh. Of course. Take your time."
    hide mika with dissolve
    "You step back into the cold. Behind you, the light in the window goes out."
    ending cold "Quiet End" "Just Looking"
```

1. Save with Ctrl+S.

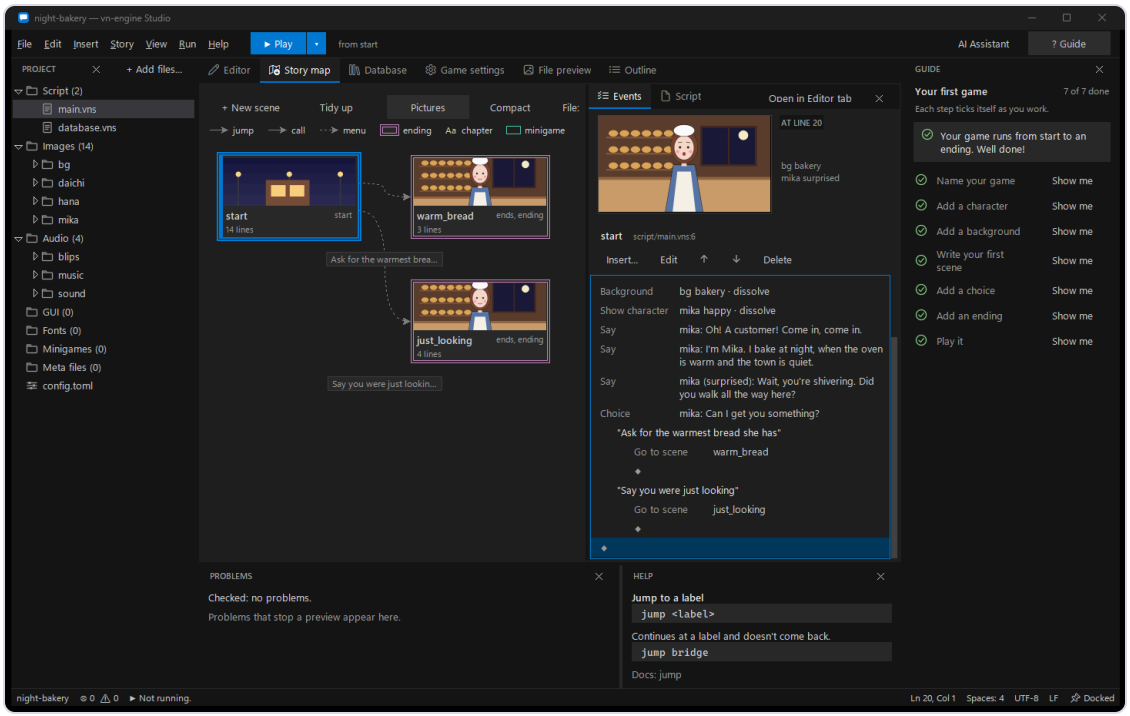
`ending warm "Good End" "A Warm Loaf"` names the ending `warm`, with a small heading and a title for the card.

You should see "Checked: no problems." again, and the Guide ticks "Add an ending".

## Step 12: Play to the end

1. Click the **Story map** tab.

You should see three boxes: `start`, with dotted arrows to `warm_bread` and `just_looking`. The arrows are labelled with the choice text, and the two scenes are marked "ends, ending".




The story map with three scenes. The start scene is open beside it in Events.

1. Press Shift+F5 (**Play from the start**), click **New Game**, and play. Pick a choice.

You should see the choice buttons, then the scene you picked, then the ending card.




The choice.




The ending card.

When every step is ticked, the Guide says "Your game runs from start to an ending. Well done!" You have made a game.

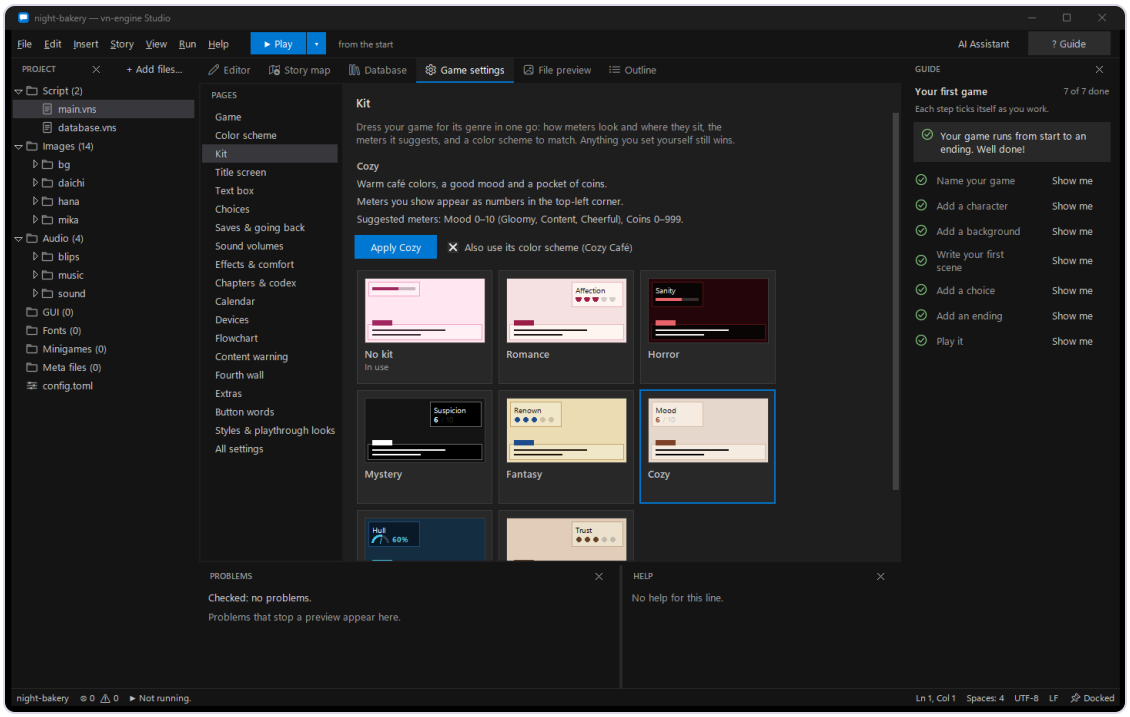
PART 2

# Make it yours

## Pick a kit and a color scheme

A **kit** dresses your game for a kind of story in one go: colors, how meters look, how the calendar reads, how letters arrive. A **color scheme** changes only the colors.

1. Click **Game settings** ▸ **Kit**. Click a card to read what it does; nothing changes yet. This guide uses **Cozy**: "Warm café colors, a good mood and a pocket of coins."
2. Leave **Also use its color scheme (Cozy Café)** ticked and click **Apply Cozy**.
3. Choose **File** ▸ **Save all**.




Game settings ▸ Kit with the Cozy card picked.

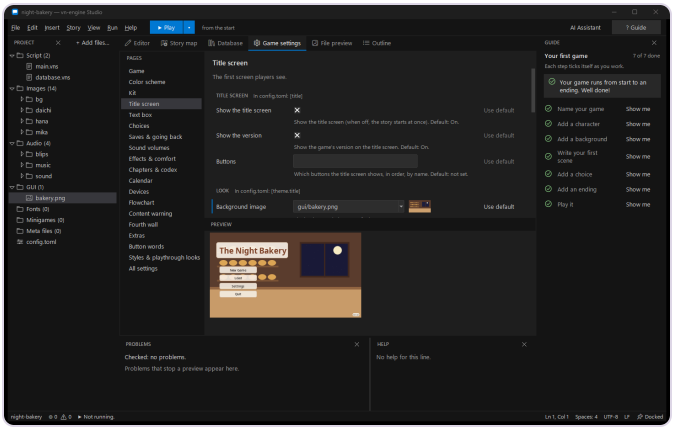
The **Color scheme** page has 28 schemes. Click a card to see the text box in those colors in the preview, then click **Apply** to use it. Play the game to see the new look everywhere.

## The title screen

The title screen's background picture comes from the **GUI** folder.

1. In the Project panel, click **GUI**, then **+ Add files...**, and pick a picture, such as `bakery.png`.
2. Click **Game settings** ▸ **Title screen**. Under **Look**, pick `gui/bakery.png` in **Background image**, and `music/lantern_walk.wav` in **Music**.
3. Save all.

You should see the preview at the bottom of the page change half a second after each change.



The Title screen page and its preview.




The title screen in the game, with the Cozy colors.

## Music and sounds

Music and sounds live in the **Audio** folder. The sample gave you `music/lantern_walk.wav` and `sound/chime.wav`. To add your own, use **Database** ▸ **Sounds** ▸ **+ New** ▸ **Add music...** (or **Add sound effects...**). OGG files work best for music.

Add these lines to your scenes:

```
play music "music/lantern_walk.wav" fadein 2.0
play sound "sound/chime.wav"
stop music fadeout 2.0
```

- `play music` starts a song that repeats until another one starts. `fadein 2.0` fades it in over 2 seconds.
- `play sound` plays a short sound once, like a door bell.
- `stop music fadeout 2.0` fades the music out.

In *The Night Bakery*, the music starts on the street, the bell rings as you walk into the bakery, and the music fades out when you leave in the `just_looking` scene.

## Transitions and effects

- `with fade` and `with dissolve` go at the end of a `scene`, `show` or `hide` line. `with dissolve 1.5` takes 1.5 seconds.
- `show mika happy at right` puts Mika on the right. `left`, `center` and `right` are ready to use.
- `shake` shakes the picture for a moment: `shake small`, `shake large`.
- `flash` covers the picture with a color that fades: `flash white`.
- `pause 1.5` waits a second and a half.

For example, after Mika's second line:

```
"Something crashes behind the counter."
shake small
```

Say what happened in the text too: some players turn shaking and flashing off in their Settings.

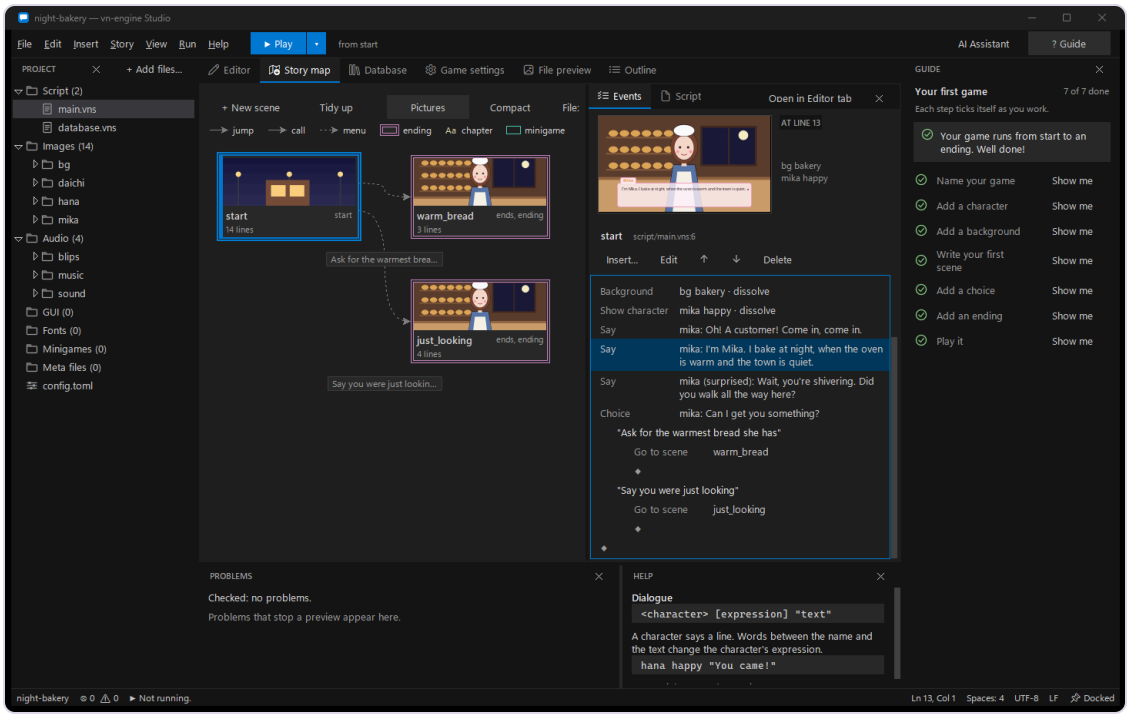
## Name colors and the text box

- **Database** ▸ **Characters** ▸ **Mika**: **Name colour**, **Text colour** (the color of her lines) and **Italic**.
- **Game settings** ▸ **Text box**: the text speed, fonts, colors and size of the box where lines appear. The preview at the bottom shows your changes.

## See the stage at any line

1. On the **Story map**, click the `start` box.
2. In **Events**, click any row.

You should see the picture above the rows change to **AT LINE** and the line's number: the stage at that moment, with the text box showing the line. Press Up and Down to flip through the scene like a storyboard. Before you click a row, the picture shows the **OPENING SHOT**: what the player sees first in that scene.




Events with a row picked: the stage at that line, above the rows.

PART 3

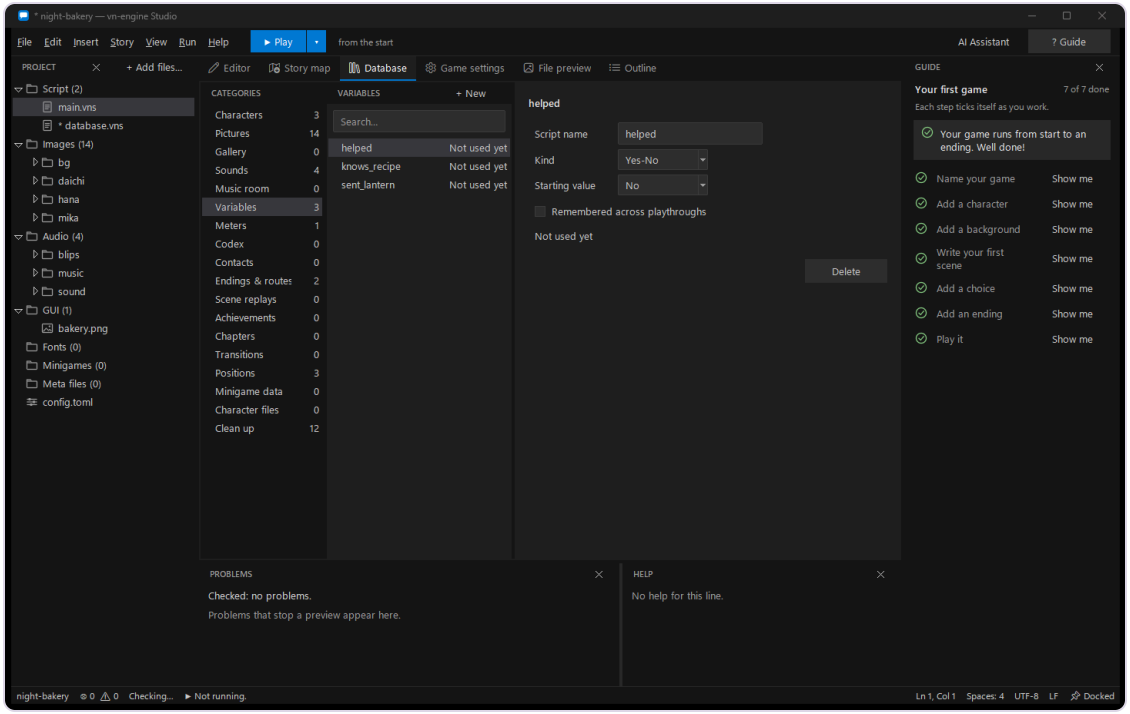
# Choices that matter

## Variables

A **variable** is something the game remembers during a playthrough, such as "did the player help Mika?". It has a name and a value.

1. In the **Database**, click **Variables**, then **+ New**. Type `helped` and click **Add**.
2. In its form, set **Kind** to **Yes-No** and **Starting value** to **No**.
3. Save all.

Studio writes `default helped = False` into `database.vns`: `helped` starts as No (`False`). To change it in a scene, write a line that starts with `$`:




Database ▸ Variables ▸ helped, a Yes-No variable.

```
$ helped = True
```

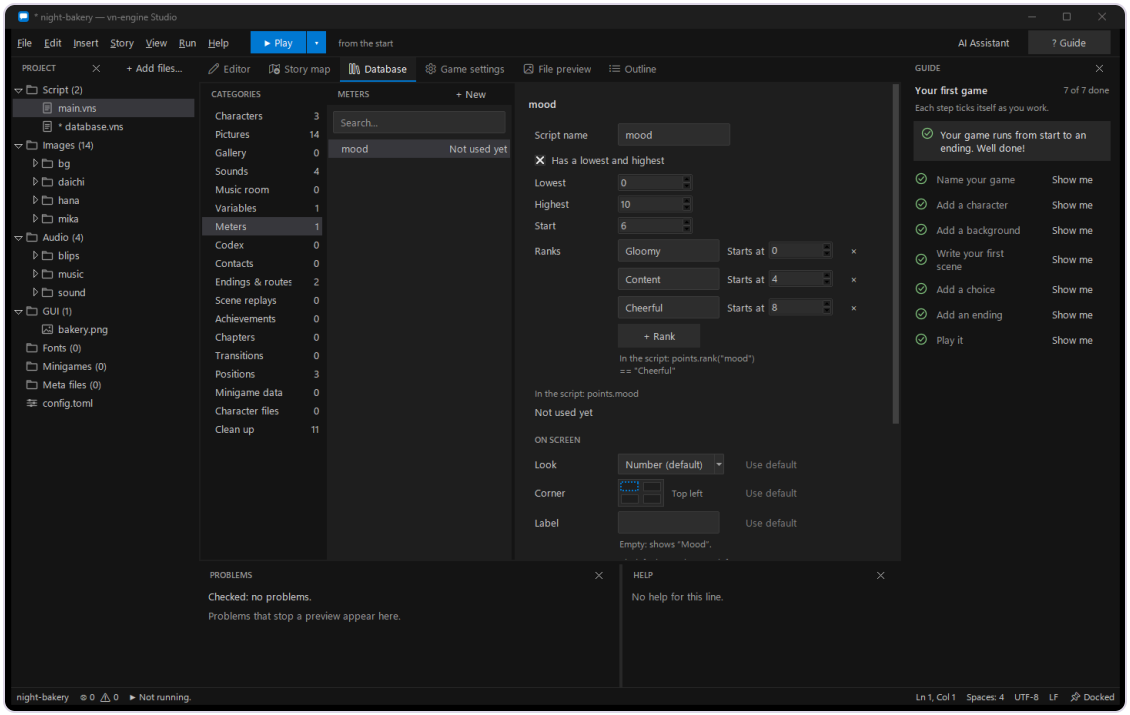
To check it, use `if`. The indented lines run only when it is true:

```
if helped:
    mika "No charge. You saved my best tray."
```

## Meters

A **meter** is a number the game keeps, such as how much a character likes the player, with a lowest and highest value. The Cozy kit suggests one called Mood.

1. In the **Database**, click **Meters**, then **+ New**. The menu shows "Suggested by the Cozy kit": pick **Mood · 0 to 10, starts at 6 · Gloomy, Content, Cheerful**.




The Mood meter in the Database, with its ranks and how it shows on screen.

The meter's **ranks** name parts of its range: below 4 is Gloomy, 4 to 7 Content, 8 and up Cheerful. Use it in your scenes:

```
show meter mood
points mood += 2 "Mika looks relieved."
```

- `show meter mood` puts the meter in a corner of the screen.
- `points mood += 2` adds 2. `-= 3` takes 3 away. The text in quotes shows as a small notice in the corner.
- `points.mood` reads it, as in `if points.mood >= 8:`.

Here is the start of *The Night Bakery* with a first small choice that changes both:

```
show meter mood
mika "Oh! A customer! Come in, come in."
mika "I'm Mika. I bake at night, when the oven is warm and the town is quiet."
"Something crashes behind the counter."
shake small
menu:
    "Help her pick up the tray":
        $ helped = True
        points mood += 2 "Mika looks relieved."
    "Wait by the door":
        $ pass
```

`$ pass` means "do nothing": a choice needs at least one line under it.




The meter in the top left corner, and the notice at the top right.

## Choices that need something

Add `if` and a test after a choice's text, and the button shows only when the test is true:

```
menu:
    mika "Can I get you something?"
    "Ask for the warmest bread she has":
        jump warm_bread
    "Offer to help her bake" if points.mood >= 8:
        jump help_bake
    "Say you were just looking":
        points mood -= 3
        jump just_looking
```

Players who helped with the tray have a Mood of 8 and see three buttons; the others see two. Keep at least one button without `if`, so there is always something to pick. In the Choice form, this is the option's **Only show when**.



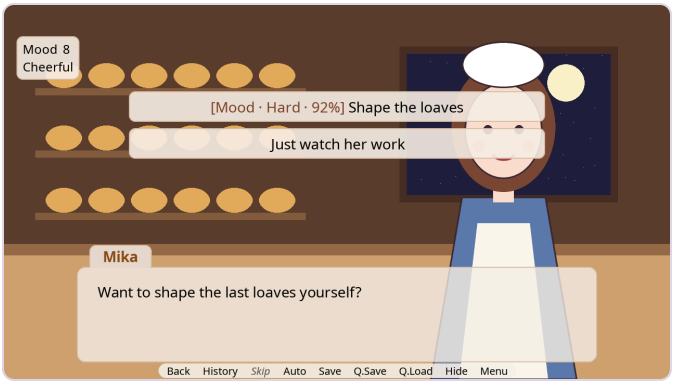

With Mood at 8, the third button shows.

## Checks: rolling the dice

A choice can roll two dice and add a meter to decide what happens. Write `check`, the meter and how hard it is (`easy`, `medium`, `hard` or a number). Under it go two parts: `success:` and `failure:`.

```
label help_bake:
    mika happy "Really? Then wash your hands and grab an apron."
    menu:
        mika "Want to shape the last loaves yourself?"
        "Shape the loaves" check mood hard:
            success:
                "The dough folds under your hands like it knows what to do."
                mika surprised "You're a natural! Let me tell you my secret recipe."
                ending apprentice "True End" "The Night Baker"
            failure:
                "The dough sticks to everything, including your hair."
                mika happy "Ha! Let's just eat the bread I made."
                jump warm_bread
        "Just watch her work":
            jump warm_bread
```

You should see the odds on the button before the player picks: "[Mood · Hard · 92%] Shape the loaves". Write the failure as part of the story, not a dead end.




The odds on a check.




Success: the True End.

## Many endings

Your game now has three endings: `warm`, `cold` and `apprentice`. Each `ending` line names a different one. **Database** ▸ **Endings & routes** lists them, with **Go to**.

## Scenes nobody can reach

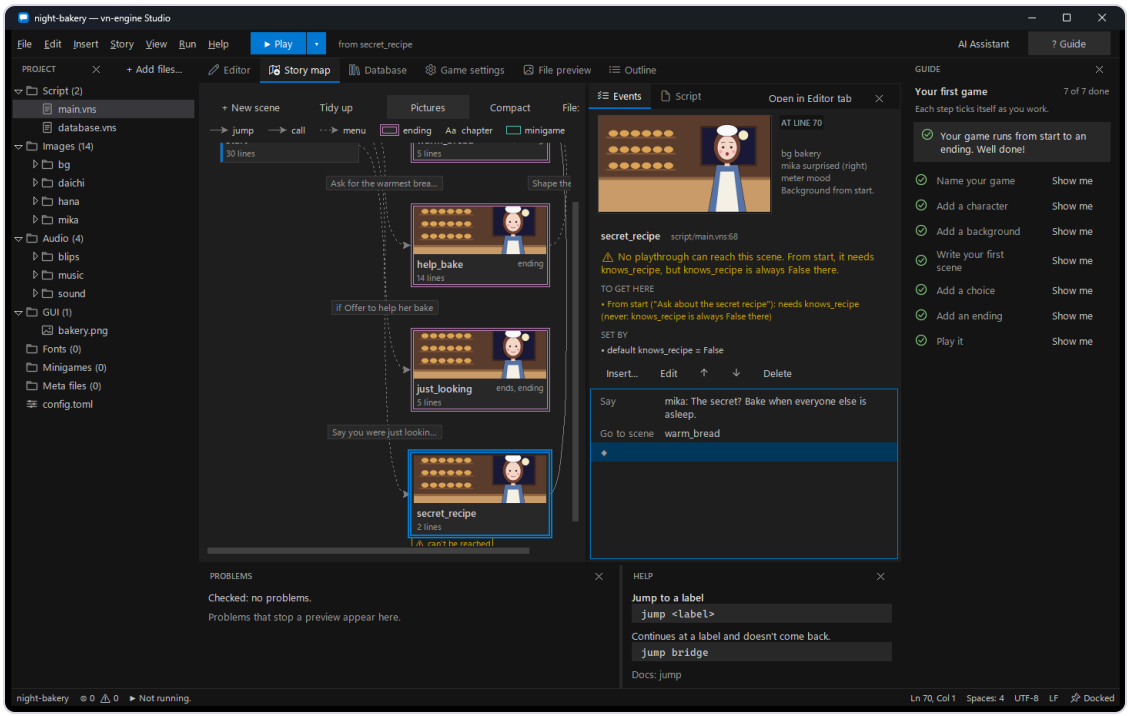
The Story map warns you about a scene that no playthrough can get to. Try it:

1. Add a Yes-No variable `knows_recipe` in the Database, starting at No.
2. Add a button to the `start` menu: `"Ask about the secret recipe" if knows_recipe:` with `jump secret_recipe` under it.
3. Add the scene at the end of the file:

```
label secret_recipe:
    mika "The secret? Bake when everyone else is asleep."
    jump warm_bread
```

1. Save, then click the `secret_recipe` box on the Story map.

You should see "⚠ can't be reached" under the box. Above the Events rows it says: "No playthrough can reach this scene. From start, it needs knows_recipe, but knows_recipe is always False there." **TO GET HERE** lists every way in and what it needs, and **SET BY** lists the lines that change it.



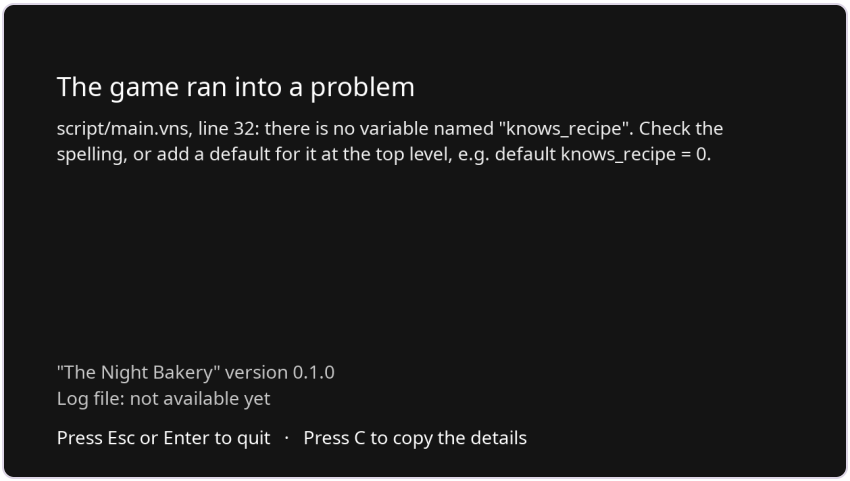

secret_recipe can't be reached. Events says why.

A scene that nothing leads to at all is marked "not reached".

## Remembering across playthroughs

To make the secret recipe show up the next time someone plays, the game has to remember it after an ending.

1. In **Database ▸ Variables ▸ knows_recipe**, tick **Remembered across playthroughs**. The form says "Kept after the game ends, in every playthrough (persistent.knows_recipe)."
2. Now change every line that uses it to say `persistent.knows_recipe`. Studio does not do this for you:




If you skip step 2, the game stops at the choice with this screen.

```
        "Ask about the secret recipe" if persistent.knows_recipe:
            jump secret_recipe
```

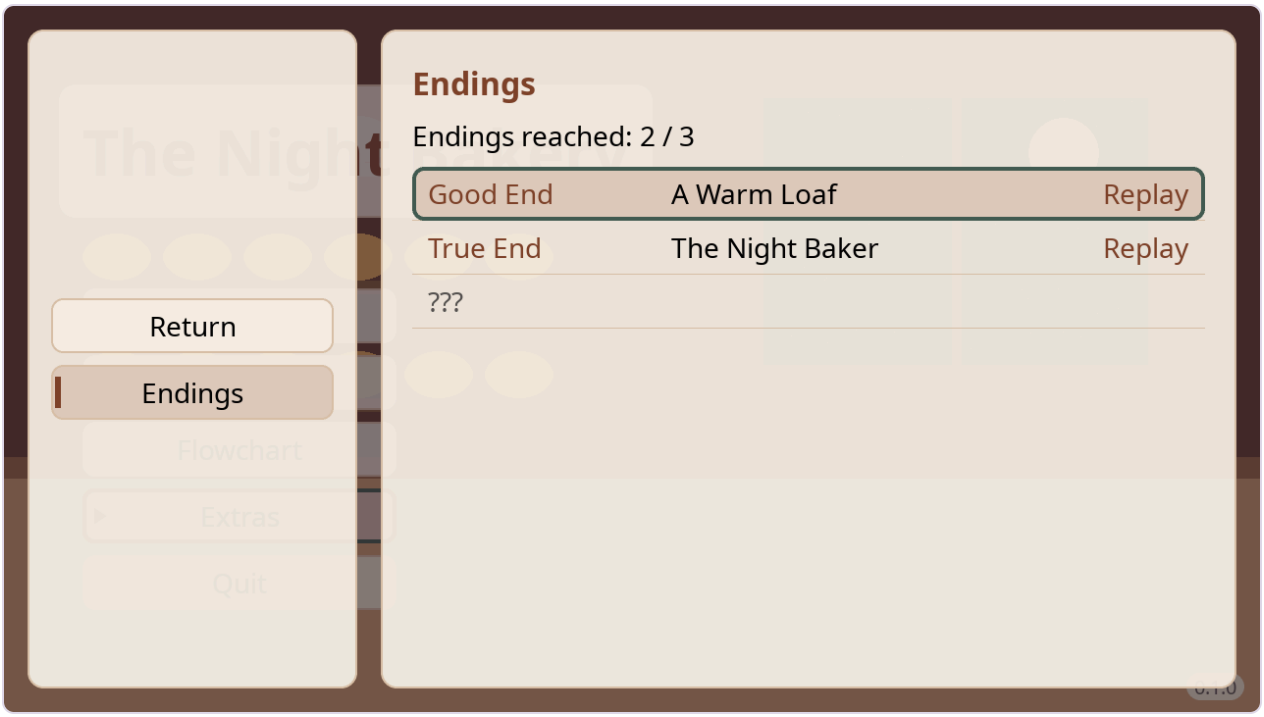
1. In `help_bake`, under the success line, add:

```
$ persistent.knows_recipe = True
```

After a player reaches the True End once, the secret recipe button shows in every new game.

## The Endings page

Once the player has reached an ending, the title screen has an **Extras** button. **Extras** ▸ **Endings** lists every ending: the ones reached with their names and **Replay**, the others as "???", and "Endings reached: 2 / 3". **Game settings** ▸ **Extras** turns the page and its Replay buttons on or off.




Extras ▸ Endings after two of the three endings.

PART 4

# Bigger games

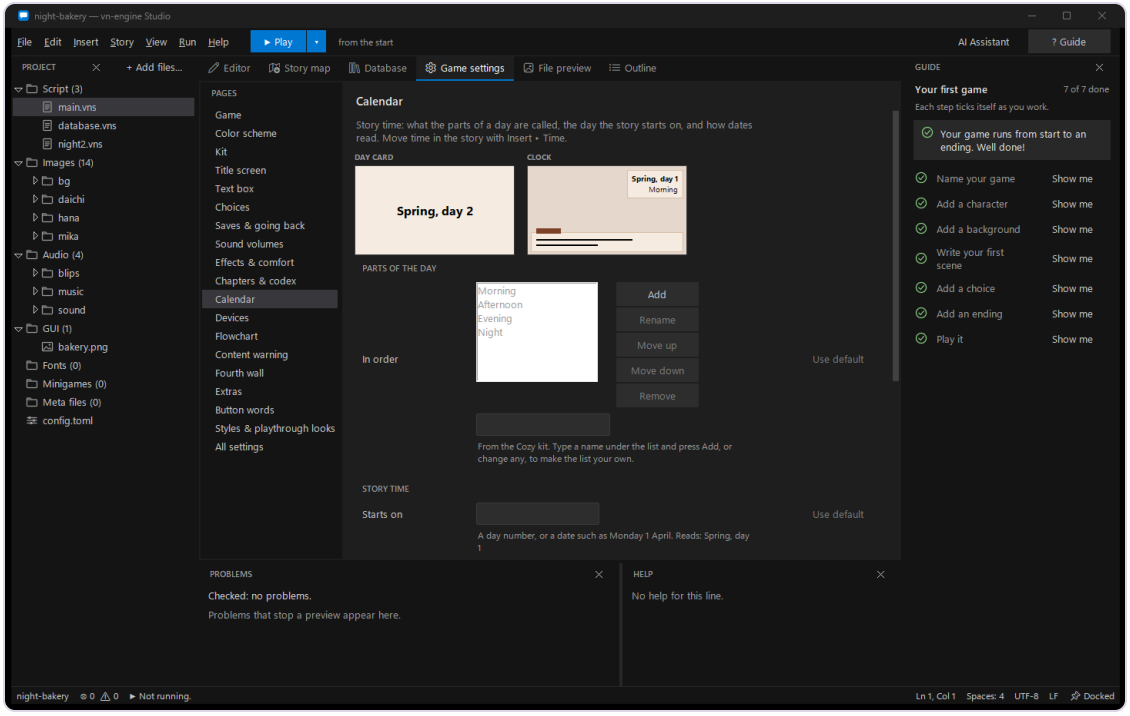
Each section shows one tool with a tiny example from *The Night Bakery*'s second night. To keep it in its own file, choose **File ▸ New script file** (Ctrl+N), type `night2.vns`, keep **Add an include line to main.vns** ticked, and click **Create**. Then add `jump night2` under Mika's "No charge" line in `warm_bread`.

## Story time: the clock and the calendar

The **story clock** counts days and parts of the day (Morning, Afternoon, Evening, Night).

```
label night2:
    clock go to day 2 Night
    scene bg bakery with fade
    show clock
```

- `clock next` moves to the next part of the day; `clock next day` to the next morning; `clock go to day 2 Night` to a day and a part.
- A new day shows a card with the date. With the Cozy kit, dates read "Spring, day 2".
- `show clock` puts the date and the part of the day in a corner.
- **Game settings ▸ Calendar** names the parts of the day, the start day, how dates read, and the clock's corner, with a preview.




Game settings ▸ Calendar with the Cozy kit's parts of the day.




show clock: the date and the part of the day in the corner.

## Items, money and a shop

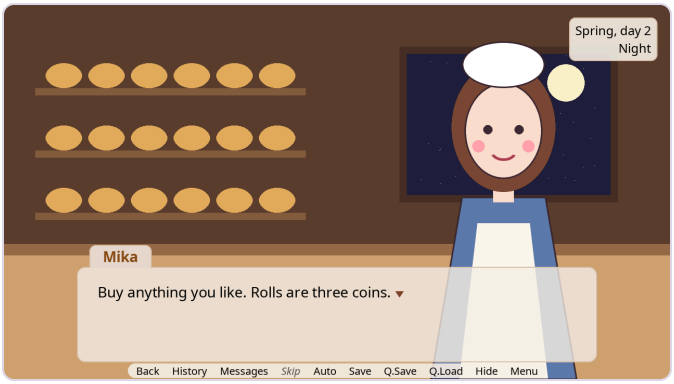
An **item** is something the player holds, kept in their bag. One item can be the player's money.

```
lock item coin "Coins" money
lock item roll "Sweet Roll" count 0 to 9 price 3
define counter_stock = {"roll": None}
default bought = {}

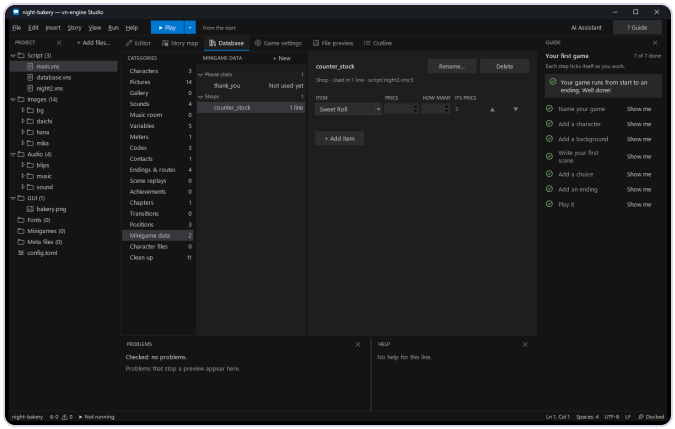
label night2:
    unlock item coin 10
    mika "Buy anything you like. Rolls are three coins."
    minigame shop stock=counter_stock name="Mika's Counter" into bought
```

- `lock item` lines go at the left edge, outside any scene. They make an item the player doesn't have yet. `count 0 to 9` lets them hold up to 9; `price 3` is its price in a shop.
- `unlock item coin 10` gives 10 coins.
- The `minigame shop` line opens the shop. `None` in the stock means "use the item's own price".

You should see a panel called "Mika's Counter" with "Coins ×10" and "Sweet Roll · 3 Coins", and **Buy**, **Put back** and **Leave**. In Studio, **Database** ▸ **Codex** lists items with their **Price**, and **Database** ▸ **Minigame data** ▸ **Shops** edits the stock as a table.




The shop in the game.



The same stock in Database ▸ Minigame data.

## The Inbox: letters and messages

**Contacts** are people who write to the player. Their messages arrive while the story goes on and stay in the Inbox. With the Cozy kit, the Inbox button is called **Messages**.

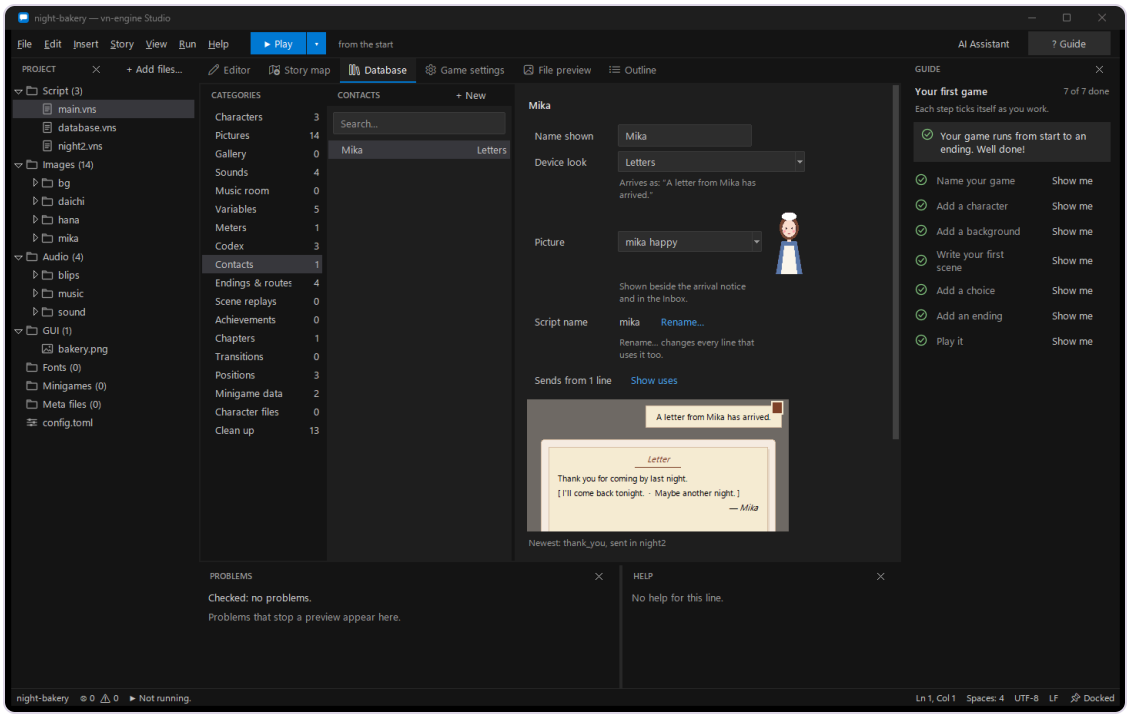
```
lock contact mika "Mika" on letters image mika happy
define thank_you = [
    "Thank you for coming by last night.",
    {"ask": "again", "yes": "I'll come back tonight.", "no": "Maybe another night."},
    ]

label night2:
    send thank_you from mika after 1 scene
    jump walk_home

label walk_home:
    scene bg street_night with dissolve
    "On the way home, a letter is waiting in your pocket."
    open inbox
    if inbox.reply("thank_you") == "yes":
        "You already know where you'll be tonight."
    ending regulars "Good End" "Regulars"
```

- `lock contact` says who writes and how: `on letters` (also `phone`, `pigeon`, `radio`, `terminal`, `mind`).
- A message is a `define` with its lines. The last line can ask a question with up to four replies.
- `send … from mika` delivers it. `after 1 scene` holds it until the story moves to another scene; `after 1 day` waits for the clock.
- `open inbox` stops the story at the Inbox until the player closes it.
- `inbox.reply("thank_you")` is the reply the player picked: `"yes"`, `"no"`, or `""` before they answer.

You should see "A letter from Mika has arrived." in the corner, and "Messages (1)" in the quick menu at the bottom. **Database ▸ Contacts** shows Mika with a preview of her newest letter, and **+ New** makes a contact without typing.




Database ▸ Contacts ▸ Mika, with a preview of her letter.




Mika's letter in the game menu's Messages page.

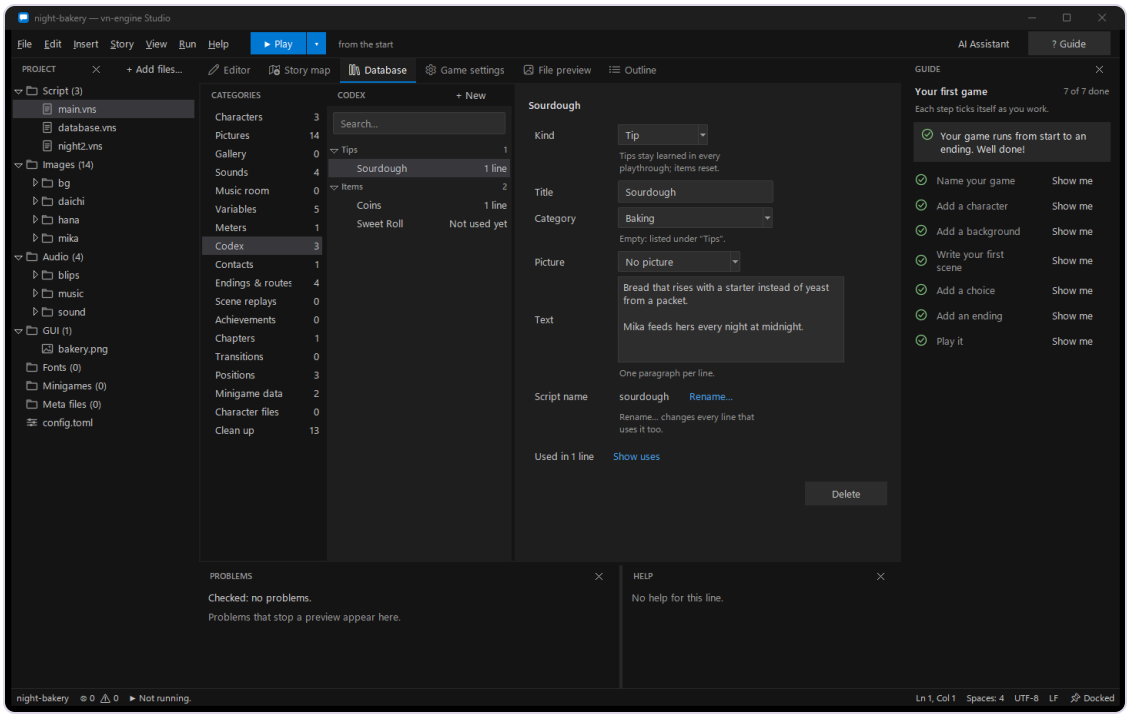
## The codex: tips and items

The **codex** is a page in the game menu with what the player has learned (**tips**) and holds (**items**). Tips are kept in every playthrough; items belong to one playthrough.

```
lock tip sourdough "Sourdough" in "Baking":
    "Bread that rises with a starter instead of yeast from a packet."
    "Mika feeds hers every night at midnight."

label night2:
    mika "Here's a word for your notebook: sourdough."
    unlock tip sourdough
```

`unlock tip` adds it and shows "New tip" in the corner. **Database** ▸ **Codex** has a form for each entry, with its **Title**, **Category**, **Picture** and **Text**.




Database ▸ Codex ▸ Sourdough.

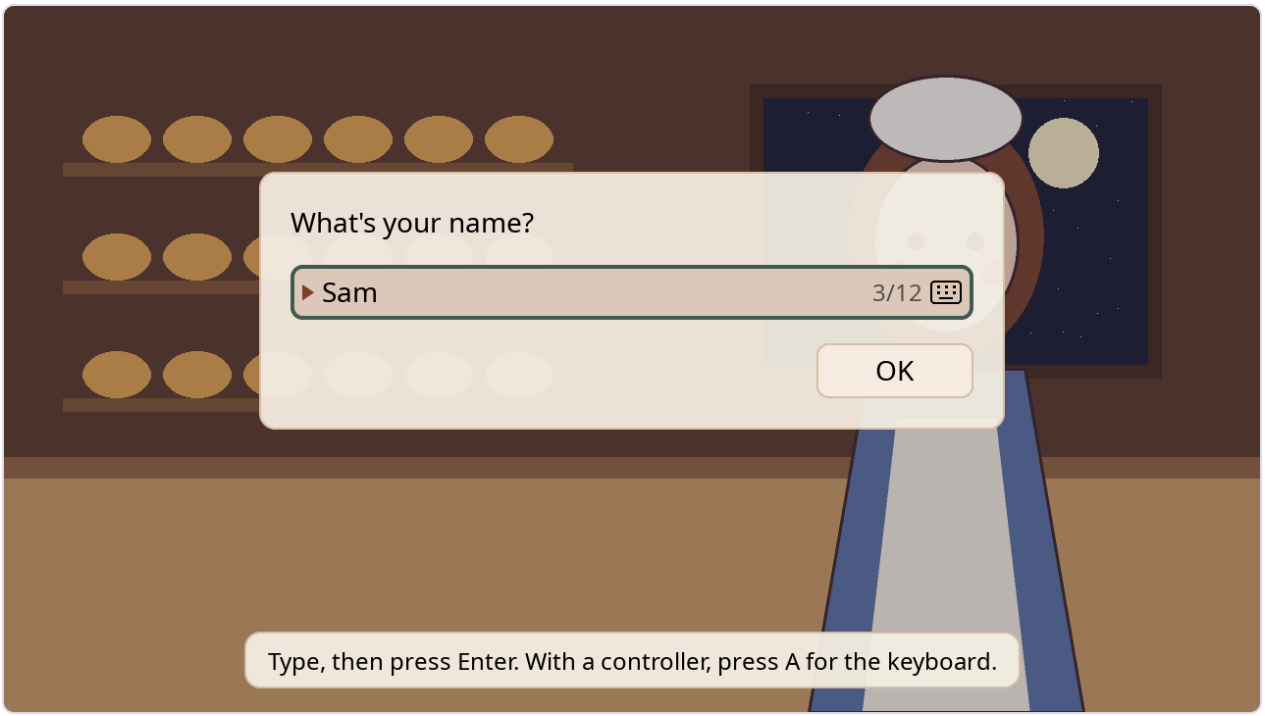
## Minigames

A **minigame** is a small game inside the story. vn-engine comes with several, such as a shop, a phone chat, a code lock and a question the player types an answer to.

```
default player_name = ""

label night2:
    mika "You came back! I never asked your name."
    minigame ask_text prompt="What's your name?" max_length=12 allowed="name" into player_name
    mika "{var=player_name}. I'll remember that."
```

`into player_name` keeps the answer in a variable, and `{var=player_name}` shows it in a line. In **Events**, **Insert...** ▸ **Ask for a name...** writes this for you. **Help** ▸ **Guides** ▸ **Minigames** lists every minigame and how to set each one up.




ask_text: the player types a name.

## Chapters

```
scene bg bakery with fade
chapter night2 "Chapter 2" "The Second Night"
```

A `chapter` line shows a title card, saves the game, and names saves after the chapter ("Chapter 2: The Second Night"). Put the `scene` line above it. **Database** ▸ **Chapters** lists them, with **Go to** and **▶ Play its scene**.

PART 5

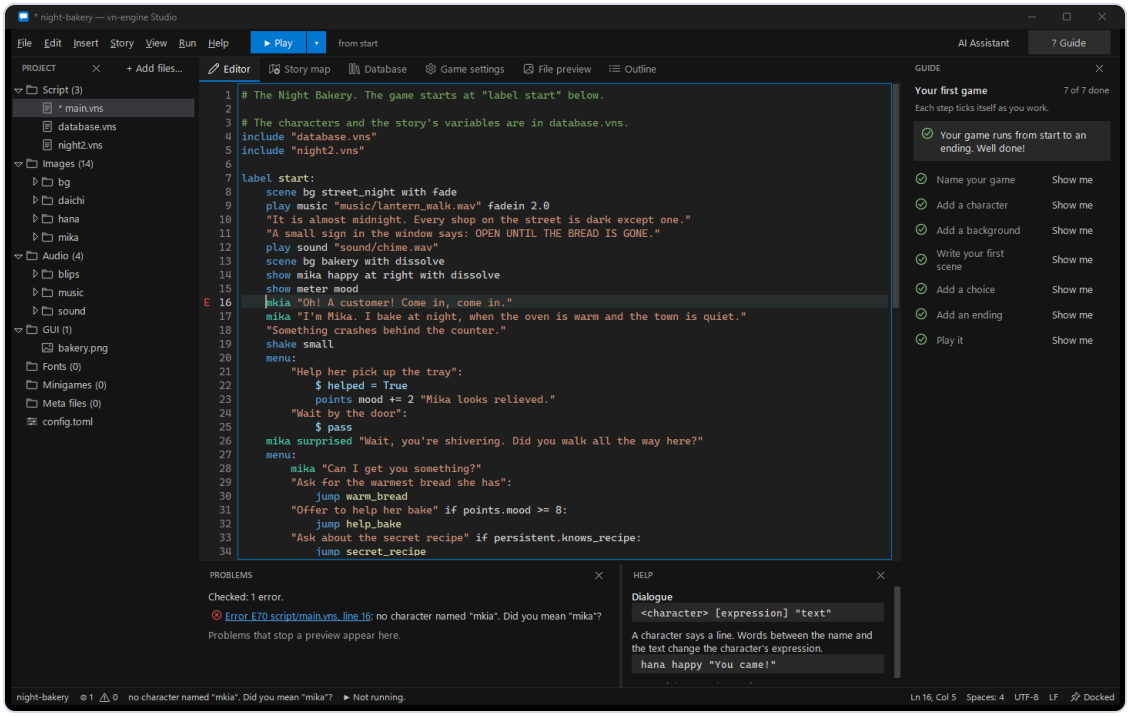
# Test and share

## The game check and Problems

Studio checks your story as you type, when you save, and when you press F7 (**Run** ▸ **Check now**). The status bar at the bottom counts errors (⊗) and warnings (⚠).

- An **error** stops the game from starting. A red `E` marks its line in the Editor.
- A **warning** is something that will look wrong, such as a missing picture. The game still runs.

Click a row in **Problems** to go to its line. Most messages say how to fix it, and many end with "Did you mean …?".




A misspelled name: a red E beside the line, and the error in Problems with its fix.

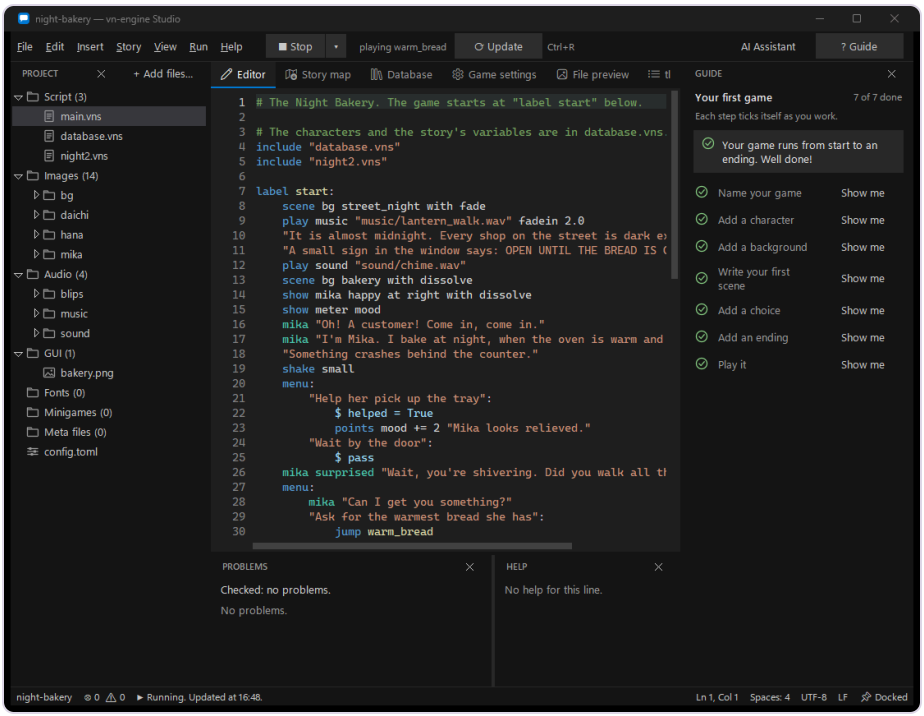
## Play from any scene

- Put the cursor in a scene and press ▶ **Play** (F5): the game starts at that scene.
- On the Story map, right-click a box and choose to play from it.
- **Run** ▸ **Play options…** has **Start values**: the values your variables should have when a scene starts this way, such as `helped = True`. Tick **Play as a new player** to play with no saves and no endings reached.

## Update while the game runs

While the game is running from Studio, change a line and save (Ctrl+S). The game closes, opens again in the same place and goes on with your new story. The status bar says "Running. Updated at" and the time. After adding new pictures or sounds, press ⟳ **Update** (Ctrl+R).

If your change has an error, the game keeps the old story until you fix it, and the status bar says so.



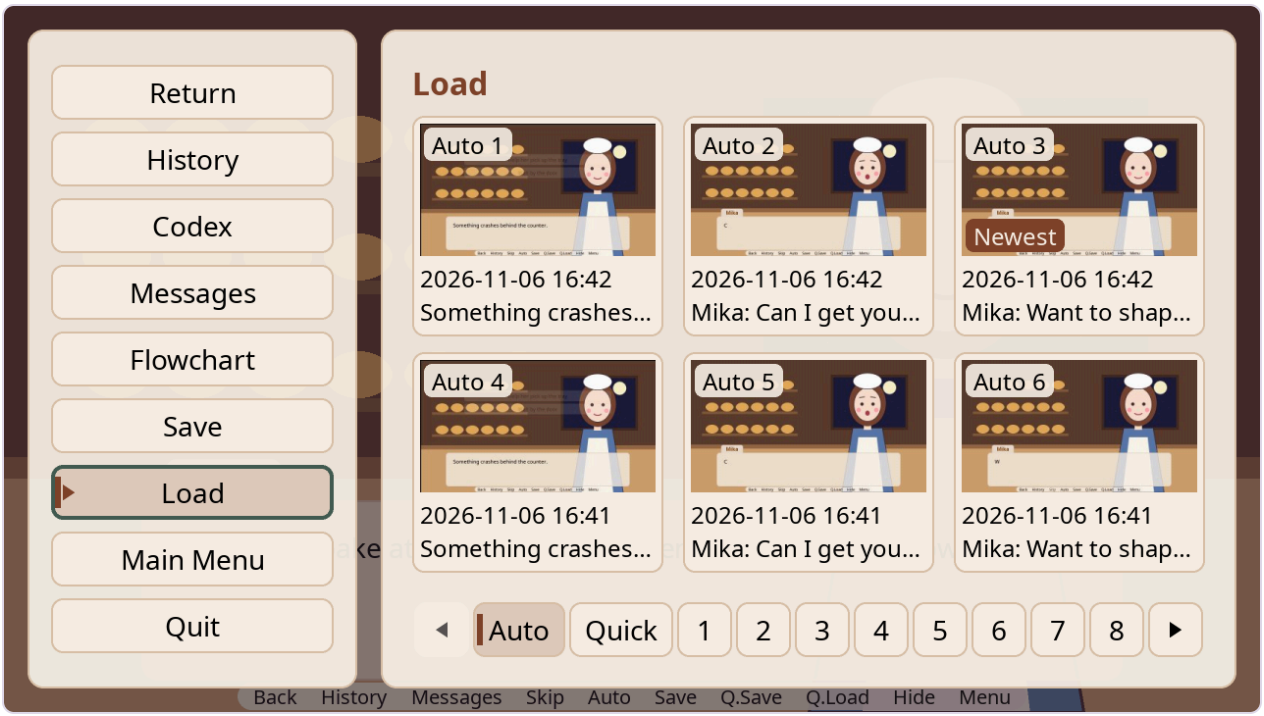

After a save, the status bar says "Running. Updated at 16:48."

## Saving and loading

Players save and load in the game, with nothing for you to write:

- The quick menu at the bottom of the game has **Save**, **Q.Save** (quick save) and **Q.Load**.
- **Esc** opens the game menu with **Save** and **Load** pages.
- The game also saves by itself at each choice, at each chapter and every few minutes.

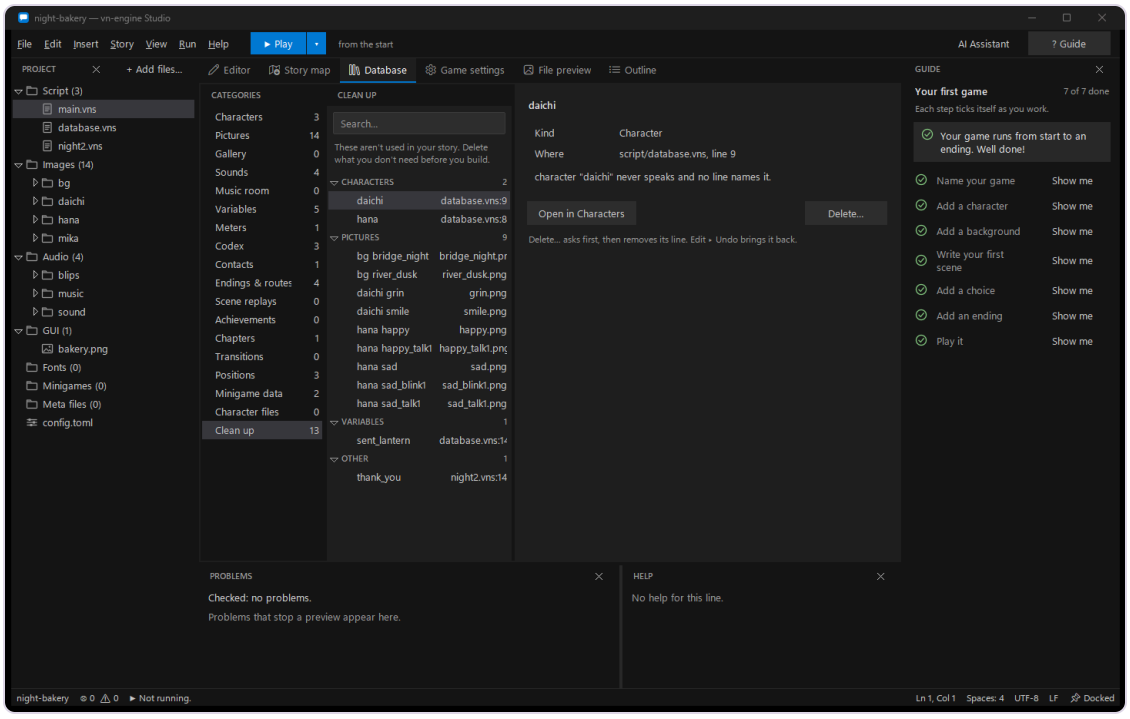
When you test from Studio, saves go in the `saves` folder inside your game folder. The built game keeps players' saves in their own folder.




The game menu's Load page, with the game's own auto saves.

## Clean up

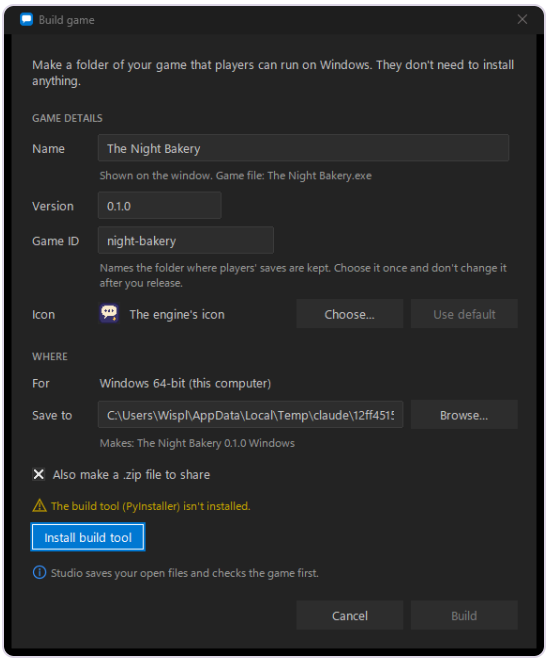
**Database** ▸ **Clean up** lists what your story doesn't use: scenes nothing leads to, characters who never speak, pictures and sounds never shown or played. In *The Night Bakery*, it lists the sample's Hana and Daichi and their pictures. Click one, then **Delete...** to remove it.




Database ▸ Clean up lists what the story never uses.

## Build the Windows game

1. Choose **File ▸ Build game...** (Ctrl+Shift+B).
2. Check the **Name** and **Version**. The **Game ID** names the folder where players' saves are kept: choose it once and don't change it after you share the game.
3. The first time, it says "The build tool (PyInstaller) isn't installed." Click **Install build tool** and wait for it to finish.
4. **Save to** is a `builds` folder next to your game folder. Keep **Also make a .zip file to share** ticked.
5. Click **Build**.



The Build game window, the first time.

You should see six steps, then "Your game is built." In the `builds` folder are a folder called `The Night Bakery 0.1.0 Windows` and a zip file. Send people the zip. They unzip it and open `The Night Bakery.exe`; they don't need Python. If Windows says "Windows protected your PC", they click **More info**, then **Run anyway**.

Studio leaves out its own files and your test saves, and says so.

TROUBLESHOOTING

# Troubleshooting

These are the mistakes a new game maker runs into most, with the exact message and the fix.

1. **Curly quotes.** "error E50: this line uses curly quotes “ ” around the text. Put the text between straight quotes "like this"." Text pasted from a word processor has curly quotes. Retype the quotes around the text in Studio. Curly quotes inside the text are fine.
2. **A misspelled name.** "error E70: no character named "mkia". Did you mean "mika"?" Fix the spelling. The name in the script is the one under "Used in the script as" in the character's form, in lower case.
3. **A scene that isn't there.** "error E43: there is no label named "warm_bred". Did you mean "warm_bread"?" Fix the name after `jump`, or make the scene.
4. **A missing colon.** "error E21: label warm_bread needs : at the end of the line. Write label warm_bread:". Lines that start a block (`label`, `menu`, a choice, `if`) end in a colon.
5. **Too much indentation.** "error E13: this line is indented more than the line above, but the line above doesn't start a block. Remove the extra indentation." Line it up with the line above.
6. **An empty choice.** "error E12: this choice has no lines under it. Indent what happens when it's picked; to just go on, write $ pass."
7. **A line outside a scene.** "error E40: this line is outside any label, so the story can never reach it. Add label <name>: above it and indent it."
8. **A picture name that doesn't match a file.** "warning W12: there is no image "mika smiling", so a placeholder will show. Add images/mika/smiling.png, or check the spelling." The look's name is the file's name without `.png`.
9. **A picture file with spaces or capitals in its name.** When you add it, Studio asks: "`Mika Happy.png` can't be an image name (spaces). Use `mika_happy.png`?" Accept, or type a better name such as `happy.png`.
10. **A remembered variable stops the game.** After ticking **Remembered across playthroughs**, the game stops with "The game ran into a problem" and "there is no variable named "knows_recipe". Check the spelling, or add a default for it at the top level, e.g. default knows_recipe = 0." Change the lines that use it to `persistent.knows_recipe`.

Two more that are easy to miss:

- **Lines after an ending never run.** "warning W130: the lines after this ending never run, because ending ends the story. Move them above the ending or into another label."
- **A sound file that isn't there.** "warning W21: there is no file audio/music/lantern_walk.mp3, so it will be silent. Add the file, or check the spelling."

GLOSSARY

# Glossary

- **Background**: a picture of a place, behind the characters. Its name starts with `bg`.
- **Block**: lines indented under a line that ends in a colon. They belong to that line.
- **Character**: someone who speaks, made with **Database ▸ Characters** (`define mika = Character("Mika")`).
- **Check**: a choice that rolls two dice and adds a meter to decide between success and failure.
- **Choice** or **menu**: buttons the player picks from.
- **Codex**: the game menu's page of tips and items.
- **Color scheme**: a set of colors for every box, menu and button in the game.
- **Condition**: a test such as `points.mood >= 8` or `helped`. With `if`, it decides whether lines run or a button shows.
- **config.toml**: the file with the game's settings. Game settings writes it for you.
- **Contact**: someone who sends the player messages in the Inbox.
- **Dialogue**: a line with a character's name before it.
- **Ending**: a line that finishes the story and is remembered across playthroughs.
- **Error** and **warning**: mistakes Studio finds. Errors stop the game; warnings don't.
- **Expression** or **look**: one picture of a character, such as `mika happy`.
- **Include**: a line that adds another story file to the game.
- **Indent**: to start a line further right, with spaces.
- **Item**: something the player holds. Items belong to one playthrough.
- **Kit**: a ready-made look for one kind of story: Romance, Horror, Mystery, Fantasy, Cozy, Sci-fi or Letters.
- **Label**: the line that starts a scene: `label warm_bread:`.
- **Meter**: a number with a range, such as Mood from 0 to 10, made with `points`.
- **Narration**: text in quotes with no name before it.
- **Persistent** or **remembered**: kept after the game ends, in every playthrough.
- **Picture tag**: the first word of a picture's name: `bg` in `bg bakery`, `mika` in `mika happy`. Showing a picture replaces the one with the same tag.
- **Playthrough**: one play of the game from New Game to an ending.
- **Rank**: a name for part of a meter's range, such as Cheerful.
- **Scene**: one part of the story, from its `label` line to the next one.
- **Script**: the story written as lines in `.vns` files.
- **Story clock**: days and parts of the day in the story.
- **Tip**: something the player learns, kept in the codex in every playthrough.
- **Transition**: how one picture changes to the next, such as `fade` or `dissolve`.
- **Variable**: a named value the game remembers during a playthrough, made with `default`.

Back to Studio.